KampConnect backend — lagen cheat sheet

Wat elke laag doet, wie naar wie mag verwijzen en waar de structuur vandaan komt

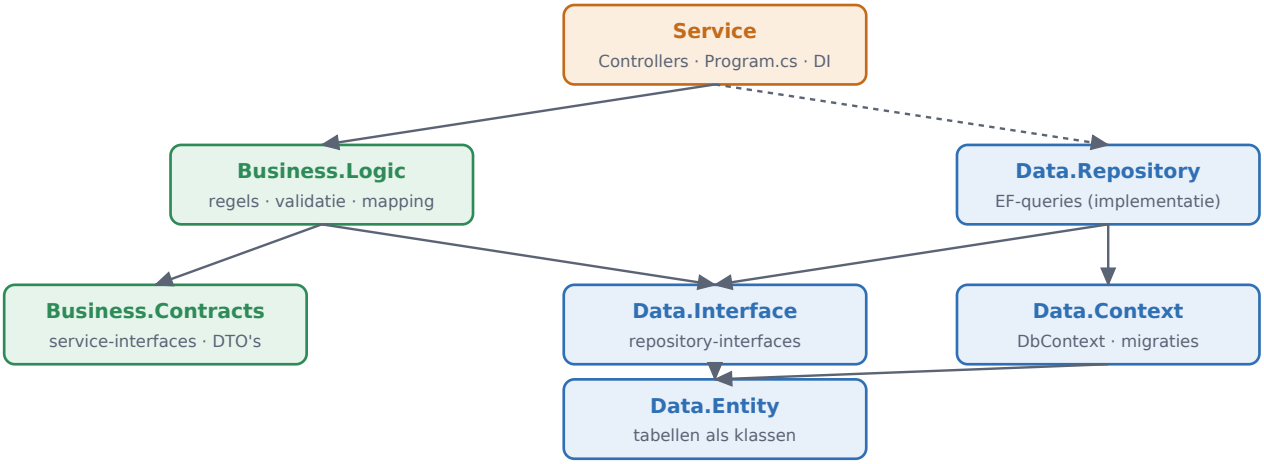
GIP 2026-27 · .NET 10 · OpenIddict

1 · Opbouw van de solution

000-Shared	Gedeelde code voor alle services: Shared.Contracts (events/DTO's tussen services), ServiceDefaults, 020-Helpers/021-Logging (KampConnect-logger).
100-Gateway	YARP-gateway: één ingang voor de app, routeert naar de services.
200-Platform	210 Identity · 220 Tenant · 230 User · 240 Compliance
300-Kamp-Communicatie	310 Camp · 320 Messaging · 330 Calling · 340 Media · 350 Notification
400-Betalingen	410 Billing · 420 Payment
800 - Hosting	Aspire AppHost: start alle services + databases lokaal samen op.
900-Tests	Eén testproject per service (xUnit).

Binnen één service (voorbeeld **210-Identity**) is de nummering altijd dezelfde: `211-Data` (Entity, Context, Interface, Repository) · `212-Business` (Contracts, Logic) · `213-Service` (Web API) · optioneel `214-Console Apps` (hulpprogramma's zoals ClientRegistration). De mappen op schijf zijn leidend; solution folders volgen het pad.

2 · Afhankelijkheden (pijl = „verwijst naar”)



Kernregel: Business.Logic kent alleen de interface van de repository, nooit de implementatie. De Service verwijst enkel naar Data.Repository/Context om ze te **registreren in DI** (stippellijn). Services verwijzen nooit naar elkaar — communicatie gaat via HTTP of events (Shared.Contracts). `scripts/Test-Structure.ps1` controleert dit.

3 · Wat doet elke laag?

Laag	Wat zit erin	Wat NIET	Voorbeeld (Identity)
Data.Entity	Eén klasse per tabel: [Table(naam, Schema)], [Key], [MaxLength], navigatie-properties, ToString() → DumpObject().	Logica, EF-queries, DTO's.	`User`, `Role`, `UserRole`, `UserToken`
Data.Context	DbContext met DbSets, OnModelCreating (indexen, OpenIddict-mapping), migraties.	Queries voor features.	`KampConnectIdentityDbContext`, map `Migrations/`
Data.Interface	Repository-interfaces: enkel wat de logica nodig heeft. Geeft entities terug.	IQueryable naar buiten, implementatie.	`IUserRepository`
Data.Repository	Implementatie met EF Core: Include, AsNoTracking, SaveChanges. CancellationToken overal.	Businessregels (lockout, validatie).	`UserRepository`
Business.Contracts	Service-interfaces, request/response-DTO's, resultaat-types en enums. Geen code met gedrag.	Referenties naar Data of EF.	`ILoginService`, `LoginResult`
Business.Logic	De regels: validatie, statusovergangen, mapping entity ↔ DTO, wachtwoord hashen, events publiceren.	HTTP, controllers, OpenIddict-specifieke zaken.	`LoginService`, `UserClaimsService`
Service	Program.cs (DI, auth, middleware), controllers/endpoints, Razor-loginpagina, ProblemDetails, appsettings.	Businessregels, directe EF-queries.	`AuthorizationController`, OpenIddict-config
Console Apps	Hulpprogramma's die dezelfde Context gebruiken: seeding, beheer.	Draait niet in productie als service.	`ClientRegistration`: clients, rollen, testgebruiker

KampConnect backend — lagen cheat sheet

GIP 2026-27 · .NET 10 · OpenIddict

Wat elke laag doet, wie naar wie mag verwijzen en waar de structuur vandaan komt

4 · Zo loopt een request door de lagen (voorbeeld: inloggen)

1. Flutter-app	Start de login met PKCE → opent /oauth2/authorize.
2. Gateway (YARP)	Stuurt het request door naar de Identity-service.
3. Service	Loginpagina ontvangt e-mail + wachtwoord en roept ILoginService aan.
4. Business.Logic	LoginService: gebruiker ophalen, lockout-regels, wachtwoord controleren, LoginAttempt loggen.
5. Data.Interface → Repository	IUserRepository.GetByNormalizedEmailAsync → EF-query.
6. Data.Context → SQL Server	DbContext vertaalt naar SQL op database kampconnect_identity.
7. Terug naar Service	Logic bouwt een ClaimsIdentity (sub, email, roles, tenant_id); de controller zet de OpenIddict-destinations en doet SignIn → OpenIddict geeft code en tokens.

5 · Standaard workflow per feature (van onder naar boven)

1. **Contract checken** — wat zegt api/<service>.openapi.yaml over dit endpoint?
2. **Entity** aanmaken of aanpassen (EntityTemplate.cs als vertrekpunt).
3. **Context**: DbSet + indexen in OnModelCreating → migratie maken en nalezen.
4. **Interface**: enkel de methodes die de logica nodig heeft.
5. **Repository**: de EF-implementatie.
6. **Contracts**: DTO's + service-interface.
7. **Logic**: de regels + DI-extension (AddXxxBusiness).
8. **Service**: endpoint, [Authorize]-policy, DI-registratie.
9. **Gateway**: YARP-route als het een nieuw pad is.
10. **Tests**: unit tests op Logic, integratietest op het endpoint.
11. **Afronden**: OpenAPI vergelijken, commit, Flutter-model.

Werk in **verticale slices**: één feature volledig af, niet eerst alle entities en dan alle repositories. **Definition of Done:** migratie nagelezen · tests groen · OpenAPI klopt · autorisatie getest · keuze genoteerd voor de GIP-bundel.

6 · Afspraken en conventies

Tabellen	Prefix sec_, snake_case kolommen, schema per domein (users, roles, claims, oidc, …). Tijden altijd in UTC (creation_timestamp / modification_timestamp).
Build	Directory.Build.props = gedeelde build-instellingen (net10.0, Nullable, ImplicitUsings). Niet herhalen in .csproj.
Packages	Directory.Packages.props = alle versies op één plek (Central Package Management). In .csproj enkel <PackageReference Include="…" /> zonder versie.
DI	Eén extension method per laag (bv. AddIdentityData(), AddIdentityBusiness()); Program.cs roept die enkel aan.
Async	Elke repository- en servicemethode async met CancellationToken.
Logging	KampConnect-logger uit 000-Shared: services.UseKampConnectLogger(…) met sectie Logging in appsettings.
Beslissingen	Elke structuurwijziging → nieuwe ADR in docs/adr/.
Nieuwe service	scripts/New-Service.ps1 maakt de 7 projecten + testproject aan; scripts/Test-Structure.ps1 controleert de regels.

7 · Waar komt deze structuur vandaan?

Gebaseerd op de **Activak-backend** (ADR 0001), met deze aanpassingen:

Activak	KampConnect
Business.Shared	Business.Contracts (interfaces + DTO's)
Logica in één Business-project	Apart Business.Logic-project
Data.Store (AutoMapper-laag)	Geschrapt — mapping gebeurt in Business.Logic
Interfaces verspreid	Interfaces op één vaste plek: Data.Interface en Business.Contracts
Logbestanden/client-libraries in de API	Niet meer in het API-project
Eigen OAuth-tabellen (clients, codes, tokens, signing keys)	OpenIddict beheert die in schema oidc; eigen tabellen enkel voor users, rollen, claims

8 · Handige commando's (vanuit de root van de solution)

```
dotnet ef migrations add <Naam> -p
"200-Platform\210-Identity\211-Data\KampConnect.Identity.Data.Context\KampConnect.Identity.Data.Context.csproj" -s
"200-Platform\210-Identity\213-Service\KampConnect.Identity.Service\KampConnect.Identity.Service.csproj" -o Migrations
dotnet ef database update -p <Context.csproj> -s <Service.csproj>
dotnet user-secrets set "Registration:TestUser:Password" "…" --project <ClientRegistration-map>
dotnet list <project.csproj> package --include-transitive
```

Tip: -p = project met de DbContext (daar komen de migraties), -s = startup-project (leest appsettings en de connection string).